INTEGRAL DRAGON

MODULE 02 · WORKSHOP 05

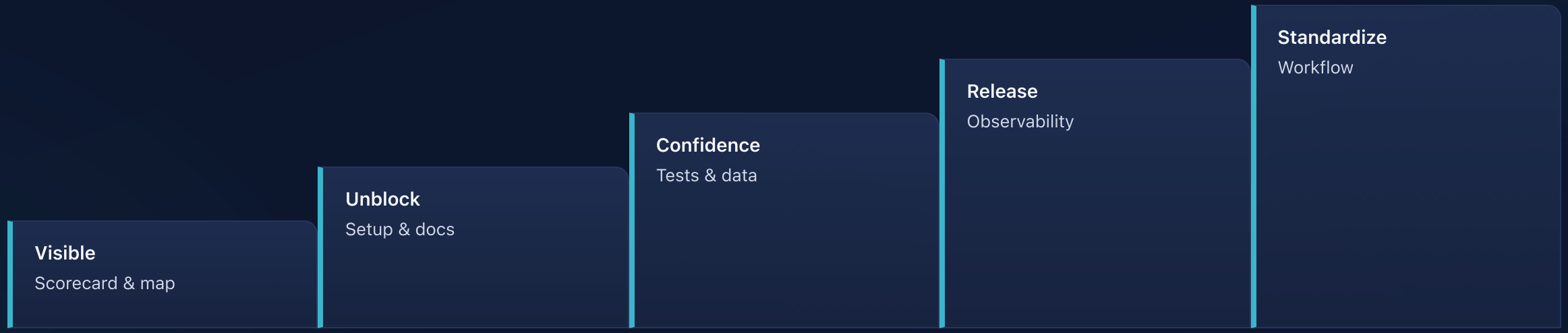
Making It Repeatable

Staged modernization over rewrite fantasies. Turning one-off cleanup into a repeatable engineering practice.

AI-ACCELERATED SOFTWARE DEVELOPMENT | APPLIED PRACTICE

Do not turn this into a rewrite fantasy

- **Most teams cannot rewrite safely or quickly** — the business cannot absorb a multi-quarter freeze.
- **Small, compounding improvements beat grand clean-slate plans** — each improvement lowers the cost of the next.
- **Start where friction is high and validation is possible** — not where the code looks ugliest.



A practical improvement sequence

STEP 01
Visible
Make the system visible — scorecard, map, debt inventory.

→

STEP 02
Setup & Docs
Fix setup and documentation blockers that stop every new change.

→

STEP 03
Confidence
Increase confidence with tests and data discipline.

→

STEP 04
Observability
Improve observability and release visibility.

→

STEP 05
Standardize
Standardize workflow and review norms so it lasts.

This is the operational shape of the course thesis: see, reduce, and systematically pay down the friction that makes software hard to change.

What to standardize in team workflow

- **Repo health scorecard reviews** · run on a regular cadence, not only when something breaks.
- **PR templates for AI-assisted work** · require scope, evidence, and validation plan in every change.
- **Small-diff preference** · narrow scope before merge; reject sprawling, speculative diffs.
- **Baseline checks in CI** · tests, lint, security scans, dependency review on every PR.
- **Periodic debt review** · a recurring prioritization session so debt does not silently accumulate.

Governance can be lightweight and still useful. The goal is normal workflow, not a program office.

What to measure

CHANGEABILITY

Onboarding friction

Time from repo clone to running the system locally or in an isolated environment.

CHANGEABILITY

Time to first safe change

How long a new engineer takes to land a reviewed, merged PR.

DELIVERY

Build reliability

Share of CI runs that go green without retries or flake-driven reruns.

CONFIDENCE

Test stability

Flake rate on main. Tests that fail randomly teach the team to ignore signal.

OPERATIONS

Deployment verification

Time from deploy to "known healthy" — not just "pipeline green".

OPERATIONS

Incident diagnosis speed

Time from alert to probable cause. Observability pays off here.

Also track **recurring friction categories** — what keeps coming up in retros. Tie every metric to practical usefulness; avoid vanity.

30-day team plan

3 × QUICK WINS

Visible, low-risk, fast

README fixes, one pre-commit hook, a broken link purge, a missing runbook — finished inside a week.

2 × STRUCTURAL

Foundations that compound

Reproducible setup, a real test for the riskiest path, an architecture map, or a schema-migration pattern.

1 × WORKFLOW

One norm everyone adopts

New PR template, small-diff rule, AI-assisted review checklist, or Conventional Commits adoption.

1 × VALIDATION

One new signal of truth

Deployment verification check, structured logging upgrade, post-deploy smoke test, or dependency scan gate.

Plus **one 30-day check-in** — rescore the repo, re-rank the debt list, or revisit one chosen metric. Force prioritization. Teams do not need 30 actions; they need the right seven.

The mature use of AI in engineering

- **Use AI to inspect, explain, propose, and standardize** — not to generate more code than the team can validate.
- **Require human review and evidence** before any AI-assisted change lands on main.
- **Treat AI output as draft material** with a burden of proof — fluency is not evidence.
- **Prefer repeatable prompts and checklists** over ad-hoc improvisation in every PR.

AI belongs inside disciplined engineering — not as a chaotic side channel that routes around review.

Final takeaways

TAKEAWAY 01

Fundamentals make AI useful

Good engineering fundamentals do not compete with AI. They make AI more powerful.

TAKEAWAY 02

Debt is friction

Technical debt is friction that compounds with every change — not just messy code.

TAKEAWAY 03

AI's real strength

AI is strongest in understanding and improvement work: inspect, summarize, scaffold, propose.

TAKEAWAY 04

Validation makes it real

Tests, observability, and security checks turn plausible output into trustworthy change.

TAKEAWAY 05

Small beats heroic

Small repeatable improvements beat heroic cleanups and rewrite fantasies every time.

NEXT

Applied Phase 3

Take this method to your own repository. Pick seven actions. Run the 30-day plan.

• CLOSE OUT

The point is not more code. The point is easier change.

Staged modernization. Validated improvements. Repeatable practice. That is what makes AI-assisted engineering lasting instead of loud.